

MUSCLE PATH

コードガイド

React・JavaScriptで筋トレアプリを作りながら理解するための、
現在のコードに対応した学習ドキュメントです。

スマホ閲覧版 / 2026-08-04

目次

MUSCLE PATH コードガイド	8
現在の実装状況	8
ファイル構成	9
app/page.jsx : トップ画面	9
FeatureCard.jsx : 共通カード	10
理想の体機能	10
処理全体の流れ	11
1. "use client" と import	11
2. bodyTypes : 体型データ	11
3. useState : 変化する値	12
selectedBodyType	12
referenceImage	12
referenceImagePreview	12
選択した体型をStateとlocalStorageへ保存する関数	13
4. 参考画像を読み込む関数	14
event.target.files?.[0] ?? null	14
ファイルがない場合	15
FileReader	15
5. map() : 体型データからカードを作る	16
6. 体型を選択する処理	16
7. カード内の表示	17
8. 参考画像の入力と表示	17
9. 体型画像の保存場所	18

globals.css : 現在使っている主な見た目	19
共通色	19
体型カード全体	19
1枚のカード	19
体型画像	20
選択中のカード	20
参考画像プレビュー	20
AIチャット機能	20
1. トップページから専用ページへ移動する	21
2. app/chat/page.jsx が専用ページになる理由	21
3. ブラウザ上の状態管理を使う	22
4. チャットで使用する状態	22
入力途中のメッセージ	22
画面内の会話履歴	22
送信状態とエラー	23
5. 質問入力欄	23
6. 利用者メッセージと仮AI回答の送信処理	25
現在の配列を残して新しい要素を追加する	28
7. フォームと送信ボタン	28
8. 会話履歴を画面へ表示する	29
messages.map()	29
発言者を含むクラス名	29
9. 利用者メッセージの吹き出し	30
10. Enterでメッセージを送信する	31
11. バックエンドの返事をAIメッセージにする	31

12. LINE風の入力バー	32
13. 送信中とエラーの表示	32
14. 専用チャット画面の構成	33
AIチャットのバックエンド	33
AI用システムプロンプト	36
OpenAI API接続の準備	37
覚える用語・ 構文一覧	38
JavaScriptの基本	39
const	39
配列 []	40
オブジェクト {}	40
入れ子のオブジェクト	40
プロパティの省略記法	41
JSON	42
request.json()	42
Response.json()	42
POST	42
fetch()	43
JSON.stringify()	43
headers	44
Content-Type	44
response	44
response.json()	45
プロパティ	45

function	46
async	46
await	46
Promise	46
try · catch · finally	47
return	47
if	47
! (否定)	47
=== (厳密等価演算子)	48
三項演算子 条件 ? A : B	48
&& による条件表示	48
(OR・ または)	48
テンプレートリテラル \${}	49
アロー関数 =>	49
.map()	49
スプレッド構文	50
.trim()	51
.length	51
オプションルチェーン ?.	51
Null合体演算子 ??	51
crypto.randomUUID()	52
Reactの基本	52
コンポーネント	52
JSX	52
<a> と href	52
ルート	52
import	53

new	53
export default	54
props	54
children	54
useState	54
状態の関数更新	56
key	56
イベント	56
イベントハンドラー	56
event.target	56
event.preventDefault()	57
onKeyDown	57
event.key	57
shiftKey	57
isComposing	57
currentTarget	58
requestSubmit()	58
条件レンダリング	58
制御コンポーネント	58
className	58
disabled	59
role="alert"	59
ブラウザAPI	59
localStorage	59
保存する基本の型	59
読み取る基本の型	60
FileReader	60

onload	61
Data URL	61
CSSの基本	61
CSSクラス	61
CSS変数 var()	61
Grid	61
Flexbox	62
flex-direction	62
align-self	62
object-fit	62
:focus	62
::placeholder	63
:disabled	63
現在まだ実装していないこと	63

MUSCLE PATH コードガイド

このファイルは、過去の作業手順ではなく、現在プロジェクトにあるコードの構成と役割を説明します。

コードを変更したときは、古い説明を残して追記するのではなく、該当する説明を現在のコードに合わせて更新します。

実際のソースコードには、処理のまとまりの直前へ「何をする場所か」を示す1行コメントを付けます。1行ずつ動作を直訳するコメントは増やさず、状態・処理・表示などの役割が切り替わる場所だけに付けます。

現在の実装状況

機能	現在の状態
理想の体	画像付き体型選択、目標体型のlocalStorage保存、参考画像の選択とプレビューまで実装済み
身体分析	表示用の土台のみ
トレーニング記録	表示用の土台のみ
AIメニュー	表示用の土台のみ
ダッシュボード	表示用の土台のみ
AIチャット	専用ページからバックエンドへ質問と目標体型を送り、OpenAIの回答を取得する処理まで実装済み。画面へ返すreplyは現在まだ仮回答

現時点ではデータベースへ保存していません。目標体型はlocalStorageに残りますが、再読み込み後に選択中の見た目を復元する処理と、参考画像を保存する処理はまだありません。

ファイル構成

```

app/
├── layout.jsx
├── page.jsx
├── globals.css
├── api/
│   └── chat/
│       └── route.js
├── chat/
│   └── page.jsx
├── lib/
│   └── ai/
│       └── systemPrompt.js
└── components/
    ├── FeatureCard.jsx
    ├── IdealBodySection.jsx
    ├── BodyAnalysisSection.jsx
    ├── TrainingLogSection.jsx
    ├── MenuBuilderSection.jsx
    ├── DashboardSection.jsx
    └── AiChatSection.jsx

public/
└── images/
    └── body-types/
        ├── lean-muscle.png
        ├── v-shape.png
        ├── physique.png
        └── bulk-up.png

.env.local
└── OPENAI_API_KEYをローカル環境だけで管理する

```

app/page.jsx : トップ画面

page.jsx は、アプリのトップ画面に何をどの順番で表示するかを決めます。

```

<div className="sectionGrid">
  <IdealBodySection />
  <BodyAnalysisSection />
  <TrainingLogSection />
  <MenuBuilderSection />
  <DashboardSection />
  <AiChatSection />

```

```
</div>
```

各機能の細かい処理はここへ直接書かず、機能別のコンポーネントへ分けています。 `page.jsx` は画面全体の組み立てだけを担当します。

FeatureCard.jsx : 共通カード

`FeatureCard` は、6つの機能で共通して使うカードの外枠です。

```
export default function FeatureCard({ number, title, description, children
}){
```

受け取る値は次の4つです。

名前	用途
<code>number</code>	<code>01</code> などの機能番号
<code>title</code>	「理想の体」などの見出し
<code>description</code>	カードの説明文
<code>children</code>	カードごとに異なる中身

```
<div className="featureContent">{children}</div>
```

`children` があるため、共通の外枠を使いながら、それぞれのカードに異なる内容を表示できます。

理想の体機能

担当ファイルは `app/components/IdealBodySection.jsx` です。

このファイルは、次の3つを担当しています。

1. 4種類の体型カードを表示する
2. 選択された体型を管理する
3. 利用者が選んだ参考画像をプレビューする

処理全体の流れ

```

bodyTypesに4種類の体型データを用意
↓
map()で4枚の画像カードへ変換
↓
カードを押すとselectedBodyTypeを更新
↓
選択中のカードにselectedクラスを追加

参考画像を選択
↓
ファイルをreferenceImageへ保存
↓
FileReaderで表示用データへ変換
↓
referenceImagePreviewをimgへ表示

```

1. "use client" と import

```

"use client";

import { useState } from "react";
import FeatureCard from "../FeatureCard";

```

- **"use client"** : クリックやファイル選択など、ブラウザ上の操作を使うための指定
- **useState** : 画面上で変化する値をReactに記憶させる機能
- **FeatureCard** : 共通カードの外枠を読み込む

"use client" はファイルの先頭に置き、importはその下に書きます。

2. bodyTypes : 体型データ

```

const bodyTypes = [
  {
    name: "細マッチョ",
    image: "/images/body-types/lean-muscle.png",
    description: "体脂肪を抑えた、引き締まった体型",
  },
  // 残り3種類も同じ形
];

```

`bodyTypes` は4つのオブジェクトを持つ配列です。

1つの体型に必要な情報を、同じオブジェクトへまとめています。

プロパティ	内容
<code>name</code>	画面に表示し、選択状態にも保存する体型名
<code>image</code>	見本画像のパス
<code>description</code>	体型の特徴を説明する文章

このデータは画面操作では変化しないため、`IdealBodySection` 関数の外側に置いています。

3. `useState` : 変化する値

```
const [selectedBodyType, setSelectedBodyType] = useState("");
const [referenceImage, setReferenceImage] = useState(null);
const [referenceImagePreview, setReferenceImagePreview] = useState("");
```

`selectedBodyType`

現在選択されている体型名を保存します。

```
初期状態 : ""
選択後 : "細マッチョ"など
```

`referenceImage`

利用者が選択した元のファイルデータを保存します。

最初はファイルが存在しないため、初期値は `null` です。

`referenceImagePreview`

元の画像をブラウザで表示できる形式へ変換した文字列を保存します。

`img` 要素の `src` へ渡すための値なので、初期値は空文字です。

選択した体型をStateとlocalStorageへ保存する関数

基本の型は次のとおりです。

```
function 関数名(受け取る値) {
  Stateを更新する処理;
  localStorage.setItem("保存名", 保存する値);
}
```

実際のコードは次のとおりです。

```
// 選択した体型を画面の状態とブラウザ内の共通データへ保存する場所
function handleBodyTypeSelect(bodyTypeName) {
  setSelectedBodyType(bodyTypeName);
  localStorage.setItem("goalBodyType", bodyTypeName);
}
```

文の流れは次のとおりです。

```
bodyTypeNameで選択された体型名を受け取る
↓
setSelectedBodyTypeでReactの画面状態へ保存する
↓
localStorage.setItemで別ページからも読めるように保存する
```

- `handleBodyTypeSelect` : 体型が選択されたときに実行するための関数名
- `bodyTypeName` : `細マッチョ` など、関数へ渡される体型名
- `setSelectedBodyType(bodyTypeName)` : 選択状態を更新して画面へ反映する
- `"goalBodyType"` : localStorage内で体型を探すための保存名
- `localStorage.setItem(...)` : 同じブラウザ内に体型名を保存する

現在は関数を定義した段階です。次に体型ボタンの `onClick` からこの関数を呼び出します。

4. 参考画像を読み込む関数

```
function handleReferenceImageChange(event) {
  const selectedFile = event.target.files?.[0] ?? null;
  setReferenceImage(selectedFile);

  if (!selectedFile) {
    setReferenceImagePreview("");
    return;
  }

  const reader = new FileReader();

  reader.onload = () => {
    setReferenceImagePreview(reader.result);
  };

  reader.readAsDataURL(selectedFile);
}
```

この関数は、参考画像の選択内容が変わったときに実行されます。

event.target.files?.[0] ?? null

```
const selectedFile = event.target.files?.[0] ?? null;
```

- **event.target** : 変更されたファイル入力欄
- **files** : 選択されたファイルの一覧
- **?.[0]** : 一覧が存在するときだけ、最初のファイルを取得する
- **?? null** : 取得結果が **null** または **undefined** なら **null** を使う

通常は次の結果になります。

```
画像を選択した場合 → selectedFileにFileオブジェクトが入る
画像がない場合 → selectedFileにnullが入る
```

?. はオプションルチェンです。左側が **null** または **undefined** の場合にエラーを起こさず、**undefined** を返します。

?? はNull合体演算子です。左側が **null** または **undefined** の場合だけ、右側の値を採用します。

長く書くと、考え方は次の条件分岐と同じです。

```
let selectedFile = null;

if (event.target.files && event.target.files[0]) {
  selectedFile = event.target.files[0];
}
```

ファイルがない場合

```
if (!selectedFile) {
  setReferenceImagePreview("");
  return;
}
```

以前のプレビューを空にして、`return` で残りの処理を終了します。存在しないファイルを読み込まないための処理です。

FileReader

```
const reader = new FileReader();

reader.onload = () => {
  setReferenceImagePreview(reader.result);
};

reader.readAsDataURL(selectedFile);
```

`FileReader` は、端末から選んだファイルをブラウザで読み取る機能です。

1. `reader.onload` で、読み取り完了後の処理を先に設定する
2. `readAsDataURL` で読み取りを開始する
3. 完了後、`reader.result` をプレビュー用状態へ保存する

この処理はブラウザ内で行われます。現時点では画像を外部サーバーやAIへ送っていません。

5. map() : 体型データからカードを作る

```
{bodyTypes.map((bodyType) => (
  <button key={bodyType.name}>
    {` 画像・名前・説明 `}}
  </button>
))}
```

map() は、bodyTypes からオブジェクトを1つずつ取り出し、同じ形のカードへ変換します。

取り出した現在のオブジェクトを bodyType という名前で受け取っています。

```
bodyType.name
bodyType.image
bodyType.description
```

4枚のカードを個別に書かないため、体型を追加・修正するときは基本的に bodyTypes を変更します。

6. 体型を選択する処理

```
className={
  selectedBodyType === bodyType.name
    ? "bodyTypeButton selected"
    : "bodyTypeButton"
}
```

現在選択中の名前と、このカードの体型名を === で比較しています。

- 同じ場合 : bodyTypeButton selected
- 違う場合 : bodyTypeButton

```
onClick={() => handleBodyTypeSelect(bodyType.name)}
```

基本の型は次のとおりです。

```
onClick={() => 実行する関数(渡す値)}
```

カードをクリックすると、現在の bodyType.name を handleBodyTypeSelect へ渡します。関数内では次の2つの保存処理を行い

ます。

```
handleBodyTypeSelect(bodyType.name)
├─ setSelectedBodyTypeでStateへ保存
└─ localStorage.setItemでブラウザ内へ保存
```

Stateが変わるとReactが再表示し、選択中のクラスも切り替わります。
localStorageへも保存するため、次にチャットページから同じ体型名を読み取れます。

7. カード内の表示

```
<img
  src={bodyType.image}
  alt={` ${bodyType.name}の見本`}
  className="bodyTypeImage"
/>

<span className="bodyTypeName">{bodyType.name}</span>
<span className="bodyTypeDescription">{bodyType.description}</span>
```

- **src** : 現在の体型が持つ画像パス
- **alt** : 画像を表示できない場合や画面読み上げ用の説明
- **`${bodyType.name}`の見本** : 体型名を文章へ埋め込むテンプレートリテラル

画像・名前・説明をすべて **button** の中へ入れているため、カード内のどこを押しても選択できます。

8. 参考画像の入力と表示

```
<input
  id="referenceImage"
  type="file"
  accept="image/*"
  onChange={handleReferenceImageChange}
/>
```

- **type="file"** : 端末内のファイルを選択できる入力欄
- **accept="image/*"** : 選択対象を画像に絞る

- `onChange` : ファイル選択が変わったときに読み込み関数を実行する

```
{referenceImage && (  
  <p>選択した画像:{referenceImage.name}</p>  
)}
```

`referenceImage` が存在する場合だけ、ファイル名を表示します。

```
{referenceImagePreview && (  
  <img src={referenceImagePreview} />  
)}
```

プレビュー用データが存在する場合だけ画像を表示します。 `&&` の左側が成立した場合だけ、右側のJSXを表示する条件レンダリングです。

9. 体型画像の保存場所

体型	保存ファイル	JSXから使うパス
細マッチョ	<code>public/images/body-types/lean-muscle.png</code>	<code>/images/body-types/lean-muscle.png</code>
逆三角形	<code>public/images/body-types/v-shape.png</code>	<code>/images/body-types/v-shape.png</code>
フィジーク	<code>public/images/body-types/physique.png</code>	<code>/images/body-types/physique.png</code>
バルクアップ	<code>public/images/body-types/bulk-up.png</code>	<code>/images/body-types/bulk-up.png</code>

`public` フォルダ内のファイルは、JSXから参照するときに `public` をパスへ含めません。

globals.css : 現在使っている主な見た目

共通色

```
:root {
  --background: #0b0d0c;
  --surface: #141715;
  --border: #2c312d;
  --text: #f4f6f3;
  --muted: #9da69f;
  --accent: #b6f24b;
}
```

色へ名前を付け、複数の場所から `var(--accent)` のように再利用しています。

体型カード全体

```
.bodyTypeButtons {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}
```

4枚のカードを2列のグリッドで並べています。

1枚のカード

```
.bodyTypeButton {
  padding: 0;
  overflow: hidden;
  border: 1px solid var(--border);
  border-radius: 12px;
  text-align: left;
  cursor: pointer;
}
```

- `overflow: hidden` : 画像をカードの丸い角からはみ出させない
- `text-align: left` : 名前と説明を左揃えにする
- `cursor: pointer` : クリックできることをマウスカーソルで伝える

体型画像

```
.bodyTypeImage {  
  width: 100%;  
  aspect-ratio: 3 / 4;  
  object-fit: cover;  
}
```

4枚の画像を同じ縦横比へそろえ、カードの画像領域を埋めています。

選択中のカード

```
.bodyTypeButton.selected {  
  border-color: var(--accent);  
  background: var(--accent);  
  color: var(--background);  
}
```

Reactが追加する **selected** クラスを使い、選択中のカードだけ配色を変えています。

参考画像プレビュー

```
.referenceImagePreview {  
  width: 100%;  
  max-width: 320px;  
  height: 320px;  
  object-fit: contain;  
}
```

プレビューは最大320pxに抑え、**contain** で画像全体を縦横比を崩さず表示します。

AIチャット機能

担当ファイルは次の2つです。

- **app/components/AiChatSection.jsx** : トップページに表示するチャットへの入口

- `app/chat/page.jsx` : 実際に会話するAIチャット専用ページ

現在は、トップページから専用ページへの移動、質問の入力、会話の吹き出し、送信中・エラー表示、`/api/chat` への質問送信まで実装しています。OpenAI APIとの通信はまだ行っていません。

1. トップページから専用ページへ移動する

`app/components/AiChatSection.jsx` には、チャット処理そのものではなく、専用ページを開くリンクだけを書いています。

```
<a className="chatOpenLink" href="/chat">
  AIチャットを開く
</a>
```

- `<a>` : 別の場所へ移動するリンクを作るHTML要素
- `href="/chat"` : 移動先をAIチャット専用ページに指定する
- `chatOpenLink` : リンクをボタンのような見た目にするCSSクラス

トップページの役割は「機能を選ぶこと」、`/chat` ページの役割は「実際に会話すること」です。役割を分けることで、チャット画面が大きくなってもトップページのコードが複雑になりません。

2. `app/chat/page.jsx` が専用ページになる理由

このプロジェクトでは、`app` 内のフォルダ名がブラウザで開くURLになります。

```
app/page.jsx    → /
app/chat/page.jsx → /chat
```

そのため、`chat` フォルダの中へ `page.jsx` を作ると、`/chat` で表示される専用ページになります。

3. ブラウザ上の状態管理を使う

```
"use client";

import { useState } from "react";
```

文字入力とはブラウザ上の操作なので `"use client"` を指定し、入力内容の管理に `useState` を使用しています。

4. チャットで使用する状態

```
const [draftMessage, setDraftMessage] = useState("");
const [messages, setMessages] = useState([]);
const [isSending, setIsSending] = useState(false);
const [errorMessage, setErrorMessage] = useState("");
```

入力途中のメッセージ

- `draftMessage` : 現在入力欄に入っている、まだ送信していない文章
- `setDraftMessage` : 入力中の文章を変更する関数
- `useState("")` : 最初は未入力なので空文字から始める

`draft` には「下書き」という意味があります。後で追加する送信済みメッセージと区別するため、単なる `message` ではなく `draftMessage` という名前にしています。

画面内の会話履歴

- `messages` : 送信済みの利用者メッセージとAIメッセージの一覧
- `setMessages` : 会話履歴へメッセージを追加・更新する関数
- `useState([])` : 最初はメッセージがないため空の配列から始める

`messages` には、今後次の形のオブジェクトを追加します。

```
{
  id: 1,
  role: "user",
  content: "今日は何をすればいい？",
}
```

- `id` : Reactが各メッセージを区別するための値
- `role` : `user` または `assistant` で発言者を区別する
- `content` : 画面に表示するメッセージ本文

現在の `messages` はブラウザ画面内の一時的な履歴です。アプリ全体で使用する長期記憶は、後でバックエンドとデータベースを使って別に実装します。

送信状態とエラー

- `isSending` : AIの回答を待っているかを表す。最初は待っていないため `false`
- `setIsSending` : 待機状態を `true` または `false` へ変更する関数
- `errorMessage` : 通信に失敗した場合に表示する文章。最初はエラーがないため空文字
- `setErrorMessage` : エラー文章を変更する関数

5. 質問入力欄

```
<textarea
  id="chatMessage"
  value={draftMessage}
  placeholder="例：今日は何をすればいい？"
  onChange={(event) => setDraftMessage(event.target.value)}
/>
```

- `textarea` : 複数行の質問を入力できる要素
- `value={draftMessage}` : 入力欄の表示内容をReactの状態とつなぐ
- `placeholder` : 未入力時に質問例を表示する
- `event.target.value` : 入力欄の現在の文字列
- `setDraftMessage(...)` : 入力されるたびに状態を更新する

入力から状態更新までの流れは次のとおりです。

```
利用者が文字を入力
↓
```

```
textareaのonChangeが実行される
↓
event.target.valueから現在の文章を取得
↓
setDraftMessageで状態を更新
↓
value={draftMessage}へ最新の文章を表示
```

`value` と `onChange` の両方を使い、入力欄の値をReactで管理する書き方を制御コンポーネントと呼びます。

6. 利用者メッセージと仮AI回答の送信処理

```
async function handleSubmit(event) {
  event.preventDefault();

  const trimmedMessage = draftMessage.trim();

  if (!trimmedMessage || isSending) {
    return;
  }

  const goalBodyType =
    localStorage.getItem("goalBodyType") ?? "";

  const userMessage = {
    id: crypto.randomUUID(),
    role: "user",
    content: trimmedMessage,
  };

  setMessages((currentMessages) => [
    ...currentMessages,
    userMessage,
  ]);

  setDraftMessage("");
  setIsSending(true);
  setErrorMessage("");

  try {
    const response = await fetch("/api/chat", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        message: trimmedMessage,
        userData: {
          goalBodyType,
        },
      }),
    });

    const data = await response.json();

    const assistantMessage = {
```

```

    id: crypto.randomUUID(),
    role: "assistant",
    content:
      "現在は仮のAI回答です。バックエンド接続後に、目標や記録を踏まえて回答します。",
  });

  setMessages((currentMessages) => [
    ...currentMessages,
    assistantMessage,
  ]);
} catch {
  setErrorMessage(
    "回答を取得できませんでした。もう一度お試しください。"
  );
} finally {
  setIsSending(false);
}
}

```

この関数は `form` が送信されたときに実行されます。 `async` を付けることで、関数内で `await` を使い、時間がかかる処理の完了を待てるようにしています。

- `event.preventDefault()` : フォーム送信によるページ再読み込みを防ぐ
- `draftMessage.trim()` : 質問の前後にある空白を取り除く
- `if (!trimmedMessage || isSending)` : 空の質問と、回答待ち中の連続送信を止める
- `localStorage.getItem("goalBodyType")` : 理想の体機能で保存した目標体型を読み取る
- `?? ""` : 体型がまだ保存されていない場合は空文字を使用する
- `goalBodyType` : チャットからバックエンドへ送る目標体型を保存する変数
- `crypto.randomUUID()` : 各メッセージを区別するIDを作る
- `role: "user"` : 利用者側の発言であることを表す
- `content` : 実際の質問本文
- `setDraftMessage("")` : 送信後に入力欄を空に戻す
- `setIsSending(true)` : AIの回答待ち表示を始める

- `setErrorMessage("")` : 前回のエラーを消してから再送信する
- `fetch("/api/chat", ...)` : チャット用バックエンドへ質問を送る
- `method: "POST"` : データ送信用のPOSTリクエストにする
- `Content-Type` : 送るデータがJSONであることをバックエンドへ伝える
- `JSON.stringify()` : 質問オブジェクトを通信で送れるJSON文字列へ変換する
- `userData` : 目標・分析・記録など、利用者に関するデータをまとめるオブジェクト
- `goalBodyType` : 同名の変数を同名のプロパティへ入れる省略記法
- `await` : バックエンドからレスポンスが届くまで、この関数内の続きだけを待つ
- `response` : バックエンドから届いたレスポンス全体を保存する変数
- `response.json()` : レスポンス内のJSON本文をJavaScriptのオブジェクトへ変換する
- `data` : 変換後のJSONデータを保存する変数
- `catch` : 今後、通信でエラーが起きた場合にエラー文章を保存する
- `finally` : 成功・失敗に関係なく、最後に回答待ち状態を終了する

処理の順序は次のとおりです。

```

利用者の質問をmessagesへ追加
↓
isSendingをtrueにして「AIが回答を作成中...」を表示
↓
質問とuserDataをJSONにしてPOST /api/chatへ送る
↓
バックエンドからresponseを受け取る
↓
response.json()でJSON本文をdataへ保存
↓
data.replyをAIメッセージとしてmessagesへ追加
↓
finallyでisSendingをfalseに戻す

```

現在の配列を残して新しい要素を追加する

```
setMessages((currentMessages) => [
  ...currentMessages,
  userMessage,
]);
```

`currentMessages` には更新直前の会話履歴が入ります。
`..currentMessages` で既存メッセージを新しい配列へ展開し、最後に `userMessage` を追加しています。

```
更新前 : [メッセージA, メッセージB]
追加 : userMessage
更新後 : [メッセージA, メッセージB, userMessage]
```

Reactの状態に入っている元の配列を直接変更せず、新しい配列を作って `setMessages` へ渡しています。

7. フォームと送信ボタン

```
<form onSubmit={handleSubmit}>
  <textarea />

  <button
    type="submit"
    disabled={isSending || !draftMessage.trim()}
  >
    {isSending ? "送信中..." : "送信"}
  </button>
</form>
```

- `onSubmit={handleSubmit}` : フォーム送信時に送信関数を実行する
- `type="submit"` : ボタンをフォームの送信ボタンにする
- `disabled={isSending || !draftMessage.trim()}` : 未入力または回答待ち中は押せなくする
- `{isSending ? "送信中..." : "送信"}` : 待機状態に応じてボタンの文字を切り替える

送信後に入力欄が空になるのは、`setDraftMessage("")` で状態が更新され、`textarea` の `value={draftMessage}` にも空文字が反映されるためです。

8. 会話履歴を画面へ表示する

```
<div className="chatMessages">
  {messages.map((message) => (
    <div
      key={message.id}
      className={` chatMessage ${message.role}`}
    >
      <p>{message.content}</p>
    </div>
  ))}
</div>
```

会話履歴を入力フォームより上に置き、一般的なチャット画面と同じ「会話の下に入力欄」という順番にしています。

messages.map()

messages 配列からメッセージオブジェクトを1件ずつ取り出し、表示用の **div** へ変換します。

```
messages内のオブジェクト
  ↓ map()
画面上のメッセージ要素
```

現在処理している1件を **message** という名前で受け取り、次のプロパティを使用します。

- **message.id** : Reactの **key** に使用する識別子
- **message.role** : 利用者かAIかをCSSクラスで区別する値
- **message.content** : 画面に表示する本文

発言者を含むクラス名

```
className={` chatMessage ${message.role}`}
```

テンプレートリテラルを使い、共通クラスと発言者別クラスを組み合わせています。

```
roleがuserの場合 → chatMessage user
roleがassistantの場合 → chatMessage assistant
```

現在は `user` と `assistant` の両方を、同じ表示コードで吹き出しに変換しています。

9. 利用者メッセージの吹き出し

```
.chatMessages {
  display: flex;
  flex-direction: column;
  gap: 8px;
  margin-bottom: 16px;
}

.chatMessage {
  max-width: 85%;
  padding: 10px 12px;
  border-radius: 12px;
}

.chatMessage p {
  margin: 0;
  line-height: 1.5;
}

.chatMessage.user {
  align-self: flex-end;
  background: var(--accent);
  color: var(--background);
}
```

- `.chatMessages` : 会話全体を縦方向に並べる
- `.chatMessage` : 利用者とAIで共通する吹き出しの大きさや形
- `.chatMessage.user` : 利用者メッセージだけを右寄せし、アクセントカラーにする
- `max-width: 85%` : 長文でも吹き出しが横幅いっぱいにならないようにする

JSXが作る `className="chatMessage user"` に対して、`.chatMessage` と `.chatMessage.user` の両方が適用されます。

10. Enterでメッセージを送信する

```
function handleKeyDown(event) {
  if (event.nativeEvent.isComposing) {
    return;
  }

  if (event.key === "Enter" && !event.shiftKey) {
    event.preventDefault();
    event.currentTarget.form?.requestSubmit();
  }
}
```

- `isComposing` : 日本語変換中のEnterでは送信しない
- `event.key === "Enter"` : 押されたキーがEnterか確認する
- `!event.shiftKey` : Shiftが同時に押されていないか確認する
- `requestSubmit()` : textareaが所属するフォームの送信処理を実行する

`Shift + Enter` は条件に入らないため、textarea内で改行できます。

textareaでは次のように関数とキー操作をつないでいます。

```
onKeyDown={handleKeyDown}
```

11. バックエンドの返事をAIメッセージにする

```
// バックエンドのreplyをAI側のメッセージへ変換する場所
const assistantMessage = {
  id: crypto.randomUUID(),
  role: "assistant",
  content:
    data.reply,
};
```

- `data` : `response.json()` で読み取ったJSONデータ全体
- `data.reply` : バックエンドが `reply` という名前で返した文章
- `content: data.reply` : 返事の文章をAIメッセージの本文として保存する
- `role: "assistant"` : AI用の左側吹き出しを適用する

作成した `assistantMessage` を `messages` へ追加すると、バックエンドの返事がチャット画面へ表示されます。

12. LINE風の入力バー

`chatForm` を2列のGridにして、入力欄と送信ボタンを横並びにしています。

```
grid-template-columns: minmax(0, 1fr) auto;
```

- 1列目：残りの横幅を使う入力欄
- 2列目：内容に必要な幅だけ使う送信ボタン

`chatInput` は1行分の高さとし、丸い角を持ち、フォーカス中はアクセント色の枠線になります。`chatSendButton` は未入力時に `:disabled` の見た目へ切り替わります。

13. 送信中とエラーの表示

```
{isSending && (
  <p className="chatStatus">AIが回答を作成中...</p>
)}

{errorMessage && (
  <p className="chatError" role="alert">
    {errorMessage}
  </p>
)}
```

- `isSending` が `true` の間だけ、回答作成中の文章を表示する
- `errorMessage` が空文字ではない場合だけ、エラー文章を表示する
- `role="alert"` : 支援技術へ重要なエラー表示であることを伝える
- `.chatStatus` と `.chatError` : 通常の吹き出しと区別した色や枠を指定するCSSクラス

14. 専用チャット画面の構成

```
<main className="chatPage">
  <header className="chatPageHeader">...</header>
  <section className="chatPanel">
    <div className="chatMessages chatPageMessages">...</div>
    <form className="chatForm">...</form>
  </section>
</main>
```

- **chatPageHeader** : トップへ戻るリンク、画面名、説明を表示する
- **chatPanel** : 会話履歴と入力フォームを一つの枠にまとめる
- **chatPageMessages** : 専用ページで会話履歴が広く表示されるようにする
- **chatForm** : チャット画面の下側に入力欄と送信ボタンを置く

メッセージがまだない場合は、次の条件表示で入力例を案内します。

```
{messages.length === 0 && (
  <p className="chatEmptyMessage">
    「今日は何する？」など、気になることを入力してください。
  </p>
)}
```

messages.length は配列に入っているメッセージ数です。0 の場合だけ案内文が表示され、メッセージを送信すると消えます。

AIチャットのバックエンド

担当ファイルは **app/api/chat/route.js** です。

このファイルは、フロントエンドから **/api/chat** へ送られた質問と目標体型を受け取り、OpenAIへ送る場所です。OpenAIからの回答は取得できる形になっていますが、現在フロントエンドへ返している文章はまだ仮レスポンスです。

```
import OpenAI from "openai";
import { systemPrompt } from "../lib/ai/systemPrompt";

// 環境変数のAPIキーを使ってOpenAIと通信する準備をする場所
```

```
const openai = new OpenAI();
```

- `import OpenAI from "openai"` : インストールしたOpenAI SDKを読み込む
- `import { systemPrompt } ...` : 別ファイルで定義したAI用プロンプトを読み込む
- `new OpenAI()` : API通信に使用するOpenAIクライアントを作る
- `openai` : このあとResponses APIを呼び出すための変数
- APIキー : `.env.local` の `OPENAI_API_KEY` からサーバー側で読み取られる

```
// フロントエンドから送られたPOSTリクエストを受け取る場所
export async function POST(request) {
  const body = await request.json();

  // 利用者データから目標体を安全に読み取る場所
  const goalBodyType =
    body.userData?.goalBodyType ?? "未設定";

  // システムプロンプト・目標体・質問をOpenAIへ送り、回答を作る場所
  const aiResponse = await openai.responses.create({
    model: "gpt-5.6-luna",
    instructions: `${systemPrompt}`
  });

  # 今回の利用者データ

  目標体 : ${goalBodyType}`,
  input: body.message,
});

// 受け取った質問と目標体を含む返事をJSONで返す場所
return Response.json({
  reply: `目標体は「${goalBodyType}」ですね。質問を受け取りました : ${body.message}`,
});
}
```

- `export` : フレームワークがこの関数を実行できるように公開する
- `POST` : データをバックエンドへ送信するときに使う処理
- `request` : フロントエンドから届いたリクエスト全体
- `request.json()` : 届いたJSONをJavaScriptのオブジェクトへ変換する
- `await` : JSONの読み取りが完了するまで、この関数内の続きを待つ

- `body` : 変換された質問データを保存する変数
- `body.userData?.goalBodyType` : 利用者データがある場合だけ目標体型を読み取る
- `?? "未設定"` : 目標体型がない場合に使用する初期表示
- `goalBodyType` : AIへ渡すために読み取った目標体型
- `openai.responses.create()` : OpenAIのResponses APIへ回答の作成を依頼する
- `model` : 回答を作るAIモデルを指定する
- `instructions` : AIの役割、回答方針、今回の目標体型を渡す
- `input` : 利用者が入力した質問を渡す
- `aiResponse` : OpenAIから返された結果全体を保存する変数
- `return` : 作成した結果をフロントエンドへ返して、関数を終了する
- `Response.json()` : JavaScriptのオブジェクトをJSON形式のレスポンスにする
- `reply` : バックエンドから返す文章を入れるプロパティ名
- `${goalBodyType}` : 他機能で選択した目標体型を返事へ埋め込む
- `${body.message}` : 受け取った質問を返事へ埋め込む

例えば、次のJSONが届いた場合、質問は `body.message` で取得できます。

```
{
  "message": "今日は何をすればいい?"
}
```

`app/api/chat/route.js` という配置と `POST` という関数名によって、`POST /api/chat` の受付処理になります。

OpenAIへ質問を送る部分の基本の型は次のとおりです。

```
const 結果の保存先 = await OpenAIへ送る処理({
  使用するAIモデル,
  AIへの指示,
  利用者の入力,
});
```

処理は上から順番に進むため、`goalBodyType` を定義してから、その値を `instructions` で使用します。

```

質問のJSONをbodyへ保存
↓
目標体型をgoalBodyTypeへ保存
↓
質問・目標体型・基本指示をOpenAIへ送る
↓
結果をaiResponseへ保存
↓
JSONレスポンスをフロントエンドへ返す

```

現在の仮レスポンスは、次のような形でフロントエンドへ返ります。

```

{
  "reply": "目標体型は「細マッチョ」ですね。質問を受け取りました：今日は何をすればいい？"
}

```

AI用システムプロンプト

担当ファイルは `app/lib/ai/systemPrompt.js` です。

このファイルは、AIの役割や回答方針を一か所で管理する場所です。現在は「役割」「目的」「回答方針」「利用者データの扱い」を定義し、`route.js` からOpenAI APIの `instructions` へ渡しています。

```

// AIの役割と回答方針をまとめて管理する場所
export const systemPrompt = `
# 役割

あなたは、利用者の理想の身体づくりを支援する筋力トレーニングAIです。

# 目的

利用者の目標や記録を踏まえ、今日取るべき行動が分かる回答をしてください。

# 回答方針

- 日本語で、最初に結論を伝えてください。
- 利用者が実行できるよう、種目・回数・セットなどを具体的に伝えてください。
- 目標や記録が渡された場合は、その情報を回答へ反映してください。
- 必要な情報が不足している場合は、推測で決めずに質問してください。

```

- 痛みやけがが疑われる場合は、無理な運動を勧めないでください。

利用者データの扱い

- 理想の体型、身体分析、トレーニング記録、週間メニュー、過去の相談内容が渡された場合は、必要な情報を回答する。
 - 記録された事実と、AIによる提案や推測を区別してください。
 - 渡されていない情報を、知っているように回答しないでください。
 - 目標などの重要な情報を変更する場合は、利用者へ確認してください。
 ;

- `export` : 後で `route.js` から `systemPrompt` を読み込めるように公開する
- `const systemPrompt` : AIへ渡す基本指示を保存する変数
- バッククォート : 改行を含む長い文章を一つの文字列として保存する
- `# 役割` : AIがどのような立場で回答するかを決める
- `# 目的` : 利用者へどのような結果を返すかを決める
- `# 回答方針` : 回答の言語、具体性、データの使い方、不足情報、安全性を決める
- `# 利用者データの扱い` : 他機能の事実とAIの推測を区別し、重要情報を勝手に変更しないようにする

`route.js` へ直接長いプロンプトを書かず、別ファイルに分けることで、通信処理とAIの回答ルールを別々に読みやすく管理できます。

OpenAI API接続の準備

`openai` パッケージをインストールし、`package.json` へ依存関係として追加しています。

```
"openai": "^7.3.0"
```

インストールに使用した基本の型は次のとおりです。

```
npm install パッケージ名
```

今回のコマンドは次のとおりです。

```
npm install openai
```

- `npm` : JavaScriptのパッケージを管理するツール
- `install` : 指定したパッケージをプロジェクトへ追加する命令
- `openai` : OpenAI APIと通信するための公式JavaScriptライブラリ

APIキーはプロジェクト直下の `.env.local` で管理します。

```
OPENAI_API_KEY=
```

- `OPENAI_API_KEY` : OpenAI SDKが読み取る環境変数名
- `=` の右側 : 利用者自身がAPIキーを入力する場所
- `.env.local` : ローカル環境の秘密情報を置くファイル
- `.gitignore` の `.env*` : APIキーをGitへ登録しないための設定

APIキーはReactコンポーネント、`NEXT_PUBLIC_` から始まる環境変数、コードガイド、チャットへ書きません。サーバー側の `route.js` からだけ使用します。

覚える用語・構文一覧

この一覧には、現在のプロジェクトコードで実際に使用している、今後も繰り返し登場する用語をまとめています。

新しい構文やReactの機能をコードへ追加した場合は、その意味と使用例もこの一覧へ追加します。コードから使わなくなった用語は、一覧から削除または説明を更新します。

単語や記号だけでなく、`const [現在の値, 更新関数] = useState(初期値)` のような繰り返し使う「文の型」も、文全体の目的、読み順、値の流れに分けて説明します。

厳密には、言語として決められた書き方を「文法・構文」、Reactなどでよく使われる定番の書き方を「パターン」と呼びます。このガイドでは、理解しやすいように両方を「文の型」として扱います。

この一覧は単語帳として使います。`.map()`、`.trim()`、`useState`、`async`、`fetch()` など、別の機能でも再利用する言葉や構文がコードへ登場したら、次の内容を1項目ずつ追加します。

- 何をする言葉か
- 基本となる書き方
- 文のどこからどう読むか
- 引数として何を渡すか
- 処理後に何が返るか
- MUSCLE PATHのどこで、何のために使っているか

同じ言葉が複数のファイルに登場した場合でも、単語帳の説明は1か所へまとめ、使用場所をその項目へ追加します。

新しい構文や処理パターンの説明では、最初にアプリ固有の名前を一般的な名前へ置き換えた「基本の型」を載せます。そのあとに実際のコード、右または左からの読み方、データの流れを説明します。

```
// 基本の型
function 関数名(受け取る値){
  実行する処理
}

// 実際のコード
function handleBodyTypeSelect(bodyTypeName){
  setSelectedBodyType(bodyTypeName);
}
```

基本の型は、別の機能で同じ書き方が登場したときに再利用できる形で記録します。

JavaScriptの基本

const

後から同じ名前へ別の値を代入しない変数を作ります。

```
const bodyTypes = [];
const trimmedMessage = draftMessage.trim();
```

オブジェクトや配列の中身が絶対に変わらないという意味ではなく、変数そのものへ別の値を再代入できないという意味です。

配列

複数の値を順番に保存する入れ物です。

```
const bodyTypes = [
  { name: "細マッチョ" },
  { name: "逆三角形" },
];
```

messages も複数のメッセージを順番に保存するため、配列を使用します。

オブジェクト

関係する複数の情報を、名前付きでひとまとめにする入れ物です。

```
const userMessage = {
  id: crypto.randomUUID(),
  role: "user",
  content: trimmedMessage,
};
```

入れ子のオブジェクト

オブジェクトの中へ別のオブジェクトを入れ、関連するデータをグループにできます。

基本の型は次のとおりです。

```
{
  グループ名: {
    データ名: 値,
  },
}
```

現在のコードは次のとおりです。

```
{
  message: trimmedMessage,
```



```

    userData: {
      goalBodyType,
    },
  }

```

`message` は今回の質問、`userData` は利用者に関係する共通データです。将来は `userData` の中へ身体分析、トレーニング履歴、週間メニューなどを追加します。

プロパティの省略記法

オブジェクトのプロパティ名と、代入する変数名が同じ場合に使用できる JavaScript の文法です。

基本形は次のとおりです。

```

{
  プロパティ名: 同じ名前の変数,
}

```

名前が同じ場合は次のように省略できます。

```

{
  プロパティ名,
}

```

現在のコードでは、次の2つは同じ意味です。

```

{
  goalBodyType: goalBodyType,
}

{
  goalBodyType,
}

```

- 省略できる条件：プロパティ名と変数名が同じ
- 処理後の結果：通常の `プロパティ名: 値` と同じオブジェクト
- 現在の目的：`userData` へ `goalBodyType` を短く読みやすく追加する

JSON

フロントエンドとバックエンドの間でデータを送るために使う、文字列形式のデータです。JavaScriptのオブジェクトと似ていますが、プロパティ名をダブルクォートで囲みます。

```
{
  "message": "今日は何をすればいい?"
}
```

現在はAIへの質問を `message` という名前で送る想定です。将来は目標、会話履歴、トレーニング記録なども同じJSONへ追加できます。

`request.json()`

フロントエンドから届いたJSONを、JavaScriptで扱えるオブジェクトへ変換します。

```
const body = await request.json();
```

変換後は `body.message` のように、プロパティ名を指定して値を取得できます。

`Response.json()`

バックエンドから返したいJavaScriptのオブジェクトを、JSON形式のレスポンスにします。

```
return Response.json({
  reply: `受け取りました : ${body.message}`,
});
```

`request.json()` は「届いたJSONを読む処理」、`Response.json()` は「JSONを返す処理」です。

POST

ブラウザからバックエンドへデータを送るときに使うHTTPメソッドです。

```
export async function POST(request) {
```

現在は `POST /api/chat` へ質問を送ると、この関数が実行される構成です。

fetch()

フロントエンドからバックエンドや外部APIへ通信するためのブラウザ機能です。

基本となる文の型は次のとおりです。

```
const response = await fetch(送信先, 通信設定);
```

左から順番に読むと、次の意味になります。

`fetch(送信先, 通信設定)`

→ 指定した場所へ、指定した方法で通信を始める

`await`

→ 通信結果が届くまで、この`async`関数内の続きを待つ

`const response =`

→ 届いたレスポンス全体を`response`という変数へ保存する

- 1番目に渡す値：通信先のURL。現在は `"/api/chat"`
- 2番目に渡す値：`method`、`headers`、`body`などの通信設定
- 処理後に返る値：レスポンス全体を表す `Response` オブジェクト
- 現在の使用場所：`app/chat/page.jsx` の `handleSubmit`
- 現在の目的：利用者が入力した質問をチャット用バックエンドへ送る

JSON.stringify()

JavaScriptのオブジェクトや配列を、通信で送れるJSON文字列へ変換します。

基本となる文の型は次のとおりです。

```
JSON.stringify(JSONへ変換したい値)
```

現在のコードでは、次のオブジェクトを変換しています。

```
JSON.stringify({
  message: trimmedMessage,
```

```
})
```

- 渡す値：JSON文字列へ変換したいオブジェクト
- 処理後に返る値：JSON形式の文字列
- 現在の目的：{ message: 質問 } を `fetch()` の `body` で送れる形にする
- 反対の処理：JSONを読み取る `request.json()` や `response.json()`

headers

通信するデータについての補足情報を、名前と値の組み合わせで指定します。

```
headers: {
  "Content-Type": "application/json",
}
```

現在は、送信する `body` がJSON形式であることをバックエンドへ伝えるために使用しています。

Content-Type

通信で送るデータの種類を表すヘッダーです。

```
"Content-Type": "application/json"
```

`application/json` は「この通信の本文はJSON形式です」という意味です。

response

`fetch()` によってバックエンドから届いたレスポンス全体を保存している変数名です。

```
const response = await fetch("/api/chat", options);
```

この時点の `response` は返事の記事そのものではありません。ステータスやヘッダー、JSON本文などを含む入れ物です。次に `response.json()` を使い、JSON本文を読み取ります。

response.json()

`fetch()` で届いたレスポンスのJSON本文を読み取り、JavaScriptのオブジェクトへ変換します。

基本となる文の型は次のとおりです。

```
const data = await response.json();
```

右から順番に読むと、次の意味になります。

```
response.json()
→ レスポンスに入っているJSON本文を読み取る

await
→ 読み取りと変換が終わるまで待つ

const data =
→ 変換されたオブジェクトをdataという変数へ保存する
```

- 呼び出す対象： `fetch()` から返された `response`
- 渡す値： なし
- 処理後に返る値： JSON本文を変換したJavaScriptの値
- 現在の使用場所： `app/chat/page.jsx` の `handleSubmit`
- 現在の目的： バックエンドが返した `reply` を `data.reply` で取得できるようにする

`request.json()` はバックエンド側で届いたJSONを読み、 `response.json()` はフロントエンド側で返ってきたJSONを読みます。

プロパティ

オブジェクトが持つ個別の情報です。 `.` を使って取得します。

```
bodyType.name
bodyType.image
message.content
```

function

複数の処理をまとめ、名前を付けて再利用できるようにします。

```
function handleSubmit(event) {  
  // 送信時の処理  
}
```

async

時間がかかる処理を待てる関数にするため、`function` の前に付けます。

```
async function handleSubmit(event) {  
  // awaitを使う処理  
}
```

`async` 関数は必ず `Promise` を返します。重要なのは、待っている間にブラウザ画面全体を止めるのではなく、その関数内の `await` より後ろの処理だけを一時停止できることです。

await

`Promise` で表された処理が終わるまで、`async` 関数内の続きの実行を待ちます。

```
const response = await fetch("/api/chat", options);
```

現在は `fetch()` によるバックエンド通信が終わり、`response` を受け取るまで待つために使用しています。

Promise

「今は結果がないが、将来成功または失敗が決まる処理」を表すJavaScriptの仕組みです。API通信など、完了まで時間がかかる処理で使います。

```
fetch("/api/chat", options)
```

`fetch()` は通信結果がすぐには決まらないため`Promise`を返します。`await` を付けると、通信完了後の結果を `response` へ保存できます。

try ・ catch ・ finally

失敗する可能性のある処理を安全に扱うための組み合わせです。

```

try {
  // 成功するか失敗するか分からない処理
} catch {
  // 失敗した場合の処理
} finally {
  // 成功・失敗のどちらでも最後に行う処理
}

```

現在のチャットでは、`try` で仮回答を作り、`catch` でエラー文章を設定し、`finally` で `isSending` を `false` へ戻します。

return

関数から値を返す、または関数の処理をその場で終了します。

Reactコンポーネントでは、画面に表示するJSXを返します。

```

return <FeatureCard />;

```

条件分岐の中では、残りの処理を行わずに終了する目的でも使います。

```

if (!trimmedMessage) {
  return;
}

```

if

条件が成立したときだけ処理を実行します。

```

if (!selectedFile) {
  setReferenceImagePreview("");
  return;
}

```

! (否定)

条件を反対にします。

```

!selectedFile

```

この場合は「`selectedFile` が存在しない」という意味です。

=== (厳密等価演算子)

値とデータ型が同じか比較します。

```
selectedBodyType === bodyType.name
```

JavaScriptでは、意図しない型変換を避けるため、基本的に `==` ではなく `===` を使用します。

三項演算子 条件 ? A : B

条件によって使用する値を切り替えます。

```
selectedBodyType === bodyType.name
  ? "bodyTypeButton selected"
  : "bodyTypeButton"
```

- 条件が成立する場合 : `?` の後ろ
- 条件が成立しない場合 : `:` の後ろ

&& による条件表示

左側の値が成立する場合だけ、右側を使用します。

```
{referenceImage && (
  <p>{referenceImage.name}</p>
)}
```

Reactでは「データがある場合だけ表示する」という条件レンダリングによく使います。

|| (OR・ または)

左右どちらか一方でも条件が成立するかを調べます。

```
if (!trimmedMessage || isSending) {
  return;
}
```


この場合は「質問が空」または「AIの回答待ち中」のどちらかなら送信処理を終了します。

テンプレートリテラル `${}`

バッククォートで囲んだ文字列へ、変数の値を埋め込みます。

```
`${bodyType.name}の見本`
```

細マッチョの場合は **"細マッチョの見本"** になります。

アロー関数 `=>`

関数を短く書く構文です。

```
(event) => setDraftMessage(event.target.value)
```

このコードは、イベントを受け取り、その入力値で状態を更新する関数です。

`.map()`

配列の要素を1つずつ取り出し、それぞれを別の形へ変換した新しい配列を作ります。

基本となる文の型は次のとおりです。

```
元の配列.map((1件分の値) => 変換後の値)
```

左から順番に読むと、次の意味になります。

```
元の配列
→ 処理したいデータの一覧

.map(...)
→ 一覧を先頭から1件ずつ処理する

(1件分の値)
→ 現在処理している要素を受け取る

=> 変換後の値
→ その1件を何へ変えるか決める
```

- 呼び出す対象：配列
- `.map()` へ渡すもの：配列の各要素を処理する関数
- 処理中に受け取るもの：現在の要素。必要なら2番目に順番を表す `index` も受け取れる
- 処理後に返るもの：各要素の変換結果を並べた新しい配列
- 元の配列：書き換えられない

```
bodyTypes.map((bodyType) => (
  <button key={bodyType.name}>
    {bodyType.name}
  </button>
))
```

この例では、体型オブジェクト4件を、画面に表示するボタン4個へ変換しています。

```
元の配列：[細マッチョ, 逆三角形, フィジーク, バルクアップ]
           ↓ map()
新しい結果：[button, button, button, button]
```

`.map()` は元の配列を書き換えません。Reactで一覧を表示するときに頻繁に使います。

現在のプロジェクトでは、次の2か所で使用しています。

- `bodyTypes.map()`：体型データを画像カードへ変換する
- `messages.map()`：会話データをチャットメッセージへ変換する

スプレッド構文 ...

配列の中身を別の配列へ展開します。

```
setMessages((currentMessages) => [
  ...currentMessages,
  userMessage,
]);
```

既存メッセージを残し、その末尾へ新しいメッセージを追加しています。

`.trim()`

文字列の先頭と末尾にある空白や改行を取り除きます。

```
const trimmedMessage = draftMessage.trim();
```

空白だけのチャットメッセージが送信されることを防ぐために使用しています。

`.length`

文字列の文字数や、配列に入っている要素数を取得します。

```
messages.length === 0
```

現在は、メッセージ数が0件かを確認し、チャット開始前の案内文を表示するために使用しています。

オプションチェーン `?.`

左側が `null` または `undefined` ではない場合だけ、続きの値を取得します。

```
event.target.files?.[0]  
body.userData?.goalBodyType
```

左側の値が存在しない場合にエラーを起こさず、`undefined` を返します。

- `event.target.files?.[0]` : ファイラー一覧がある場合だけ最初のファイルを取得する
- `body.userData?.goalBodyType` : 利用者データがある場合だけ目標体型を取得する

Null合体演算子 `??`

左側が `null` または `undefined` の場合だけ、右側の値を使用します。

```
event.target.files?.[0] ?? null
```

ファイルを取得できなかった場合の値を `null` へ統一しています。

crypto.randomUUID()

重複しにくい一意な文字列をブラウザで作ります。

```
id: crypto.randomUUID()
```

Reactがチャットメッセージを区別するためのIDとして使用しています。

Reactの基本

コンポーネント

画面の一部分を担当するJavaScript関数です。名前は大文字から始めます。

```
function IdealBodySection() {  
  return <FeatureCard />;  
}
```

JSX

JavaScriptの中にHTMLに似た画面構造を書く記法です。

```
<button type="button">細マッチョ</button>
```

JSXの中でJavaScriptを使う場合は `{ }` で囲みます。

```
<p>{selectedBodyType}</p>
```

<a> と href

`<a>` は別のページや場所へ移動するリンクです。`href` へ移動先を指定します。

```
<a href="/chat">AIチャットを開く</a>
```

先頭が `/` の `/chat` は、同じアプリ内にあるAIチャット専用ページを表します。

ルート

ブラウザで開くページのURLを指します。このプロジェクトでは、`app` 内のフォルダと `page.jsx` でルートを作ります。

```
app/page.jsx    → /
app/chat/page.jsx → /chat
```

import

別のファイルやライブラリから機能を読み込みます。

代表として公開された機能を読み込む基本の型は次のとおりです。

```
import 読み込んだ機能につける名前 from "パッケージまたはファイル";
```

```
import OpenAI from "openai";
```

名前を付けて公開された機能を読み込む基本の型は次のとおりです。

```
import { 公開された機能名 } from "パッケージまたはファイル";
```

```
import { useState } from "react";
```

```
import { systemPrompt } from "../lib/ai/systemPrompt";
```

- {} なし : **export default** された代表の機能を読み込む
- {} あり : 名前付きで **export** された機能を、その名前で読み込む
- 現在のOpenAIコード : SDKは **OpenAI** 、プロンプトは **systemPrompt** として読み込む

new

クラスという設計図から、実際に利用するオブジェクトを作ります。

基本の型は次のとおりです。

```
const 変数名 = new クラス名();
```

現在のコードは次のとおりです。

```
const openai = new OpenAI();
```

右から順番に読むと、次の意味になります。

```
new OpenAI()
```

→ OpenAI APIと通信するためのクライアントを作る

```
const openai =
```

→ 作成したクライアントをopenaiという変数へ保存する

- `new` の右側：使用するクラス名
- 処理後に返る値：そのクラスから作られた新しいオブジェクト
- 現在の使用場所：`app/api/chat/route.js` のファイル上部
- 現在の目的：OpenAI Responses APIを呼び出せるようにする

export default

そのファイルの代表として、他のファイルから読み込めるようにします。

```
export default function AiChatSection() {
```

props

親コンポーネントから子コンポーネントへ渡す値です。

```
<FeatureCard
  number="06"
  title="AIチャット"
  description="トレーニングの悩みを相談する場所です。"
>
```

`FeatureCard` 側で `number`、`title`、`description` として受け取ります。

children

コンポーネントの開始タグと終了タグの間に書かれた中身です。

```
<FeatureCard>
  <p>この部分がchildren</p>
</FeatureCard>
```

useState

画面上で変化する値をReactに記憶させます。

```
const [draftMessage, setDraftMessage] = useState("");
```

これはReactで状態を定義するときの基本的な文の型です。

```
const [現在の値, 値を更新する関数] = useState(最初の値);
```

この文は、右側から左側へ考えると理解しやすくなります。

```
useState("")
```

→ 空文字を初期値にした状態をReactへ作ってもらう

```
[draftMessage, setDraftMessage]
```

→ Reactから返された「現在の値」と「更新関数」を2つの名前で受け取る

```
const
```

→ その2つの名前をこのコンポーネント内で定義する

- `draftMessage` : 現在Reactに保存されている入力途中のメッセージ
- `setDraftMessage` : 保存するメッセージを変更するための関数
- `useState("")` : 最初のメッセージを空文字にして状態を作る

この1行は「実際のメッセージを今すぐ保存する処理」ではなく、メッセージを保存・更新できる状態の入れ物を定義する処理です。

実際に新しいメッセージを保存するのは、次の更新処理です。

```
setDraftMessage(event.target.value);
```

例えば利用者が **今日は何する？** と入力すると、`setDraftMessage` が状態を更新し、`draftMessage` からその文章を取得できるようになります。

`[draftMessage, setDraftMessage]` のように、配列内の値を別々の変数で受け取るJavaScriptの文法を「配列の分割代入」と呼びます。React独自の記号ではありません。

状態を更新するとReactがコンポーネントを再表示し、画面へ最新の値を反映します。

同じ型を使って、保存する内容と初期値だけを変えています。

```
const [messages, setMessages] = useState([]);
const [isSending, setIsSending] = useState(false);
const [errorMessage, setErrorMessage] = useState("");
```

- `useState([])` : 複数のメッセージを保存するため、空の配列から始める
- `useState(false)` : 最初は送信中ではないため、`false` から始める
- `useState("")` : 最初はエラー文章がないため、空文字から始める

状態の関数更新

更新直前の状態を受け取り、次の状態を作る書き方です。

```
setMessages((currentMessages) => [
  ...currentMessages,
  userMessage,
]);
```

直前の状態を基準にメッセージを追加するため、連続した更新でも古い値を使いにくくなります。

key

Reactが一覧内の各要素を区別するための目印です。

```
key={bodyType.name}
```

`.map()` でJSXの一覧を作る場合、同じ一覧内で重複しない `key` が必要です。

イベント

クリック、文字入力、フォーム送信など、利用者の操作によって起きる出来事です。

```
onClick={...}
onChange={...}
onSubmit={...}
```

イベントハンドラー

イベントが起きたときに実行する関数です。 `handle` から始まる名前を使用しています。

```
handleReferencelImageChange
handleSubmit
```

event.target

イベントが発生した入力要素を表します。

```
event.target.value
event.target.files
```


- `value` : 文字入力欄の現在の値
- `files` : ファイル入力欄で選択されたファイル一覧

`event.preventDefault()`

ブラウザが本来行うフォーム送信処理を止めます。

```
event.preventDefault();
```

これにより、チャット送信時にページが再読み込みされるのを防ぎます。

`onKeyDown`

要素にフォーカスした状態でキーが押されたときに、指定した関数を実行するイベントです。

```
onKeyDown={handleKeyDown}
```

`event.key`

押されたキーの名前を取得します。

```
event.key === "Enter"
```

`shiftKey`

Shiftキーが同時に押されているかを表す真偽値です。

```
!event.shiftKey
```

現在は **Shift + Enter** を改行として残すために使っています。

`isComposing`

日本語などの文字変換中かを表します。変換確定のEnterでチャットが送信されることを防ぎます。

```
event.nativeEvent.isComposing
```

currentTarget

イベントハンドラーを設定した要素です。

```
event.currentTarget.form
```

現在は `onKeyDown` を設定したtextareaから、所属するformを取得しています。

requestSubmit()

フォームの通常の送信処理をJavaScriptから実行します。

```
event.currentTarget.form?.requestSubmit();
```

これにより、送信ボタンとEnter送信の両方が同じ `handleSubmit` を再利用できます。

条件レンダリング

状態やデータに応じて表示するJSXを切り替えることです。

```
{referenceImagePreview && (
  <img src={referenceImagePreview} />
)}
```

制御コンポーネント

入力欄の値をReactの状態で管理する書き方です。

```
<textarea
  value={draftMessage}
  onChange={(event) => setDraftMessage(event.target.value)}
/>
```

className

JSXでCSSクラスを指定します。HTMLの `class` に相当します。

```
className="bodyTypeButton"
```

disabled

ボタンなどを操作できない状態にします。

```
disabled={!draftMessage.trim()}
```

現在のチャットでは、未入力、空白だけ、またはAIの回答待ち中の場合に送信ボタンを押せなくします。

role="alert"

表示した内容が重要な通知であることを、スクリーンリーダーなどの支援技術へ伝える属性です。

```
<p className="chatError" role="alert">
  {errorMessage}
</p>
```

現在はAI通信に失敗した場合のエラー表示に使用しています。

ブラウザAPI

localStorage

同じブラウザ・同じアプリ内で、文字列データをページをまたいで保存できるブラウザ機能です。

保存する基本の型

```
localStorage.setItem("保存名", 保存する文字列);
```

- 呼び出す対象：ブラウザの `localStorage`
- `.setItem()` へ渡す1番目の値：後からデータを探すための保存名
- `.setItem()` へ渡す2番目の値：実際に保存する文字列
- 処理後に返る値：なし
- 現在の使用場所：`IdealBodySection.jsx` の `handleBodyTypeSelect`
- 現在の目的：選択した体型名をチャットページから読めるようにする

現在のコードは次のとおりです。

```
localStorage.setItem("goalBodyType", bodyTypeName);
```

これは「`goalBodyType` という保存名で、`細マッチョ` などの `bodyTypeName` を保存する」と読みます。

`localStorage` はこの端末・ブラウザ内だけの保存です。別端末との共有、利用者アカウントとの紐付け、身体画像や長期記憶の本格的な保存には使用しません。

読み取る基本の型

```
const 変数名 =  
  localStorage.getItem("保存名") ?? 値がない場合の初期値;
```

現在のコードは次のとおりです。

```
const goalBodyType =  
  localStorage.getItem("goalBodyType") ?? "";
```

右から順番に読むと、次の意味になります。

```
localStorage.getItem("goalBodyType")  
→ goalBodyTypeという保存名の値を読み取る
```

```
?? ""  
→ 保存値がnullまたはundefinedなら空文字を使う
```

```
const goalBodyType =  
→ 最終的な値をgoalBodyTypeという変数へ保存する
```

- `.getItem()` へ渡す値：読み取りたいデータの保存名
- 処理後に返る値：保存されている文字列。存在しない場合は `null`
- 現在の使用場所：`app/chat/page.jsx` の `handleSubmit`
- 現在の目的：理想の体機能で選んだ目標をチャット用データとして取得する

FileReader

利用者が端末から選んだファイルを、ブラウザ内で読み取ります。

```
const reader = new FileReader();
reader.readAsDataURL(selectedFile);
```

onload

FileReader の読み取りが完了したときに実行される処理です。

```
reader.onload = () => {
  setReferenceImagePreview(reader.result);
};
```

Data URL

画像などのデータを文字列として表した形式です。 **FileReader** の **readAsDataURL** で作り、 **img** の **src** へ渡せます。

CSSの基本

CSSクラス

複数の要素へ再利用できる見た目の名前です。

```
.bodyTypeButton {
  cursor: pointer;
}
```

CSS変数 **var()**

:root で定義した共通値を再利用します。

```
background: var(--accent);
```

Grid

行と列を使って要素を配置するCSSレイアウトです。

```
display: grid;
grid-template-columns: repeat(2, minmax(0, 1fr));
```

現在は体型カードを2列に並べるために使用しています。

Flexbox

要素を横方向または縦方向へ並べるCSSレイアウトです。

```
display: flex;
flex-direction: column;
```

現在はチャットメッセージを上から下へ並べるために使用しています。

flex-direction

Flexboxで子要素を並べる方向を指定します。

```
flex-direction: column;
```

column は上から下への縦並びです。

align-self

Flexbox内の特定の子要素だけ、横方向の位置を変更します。

```
align-self: flex-end;
```

現在は利用者メッセージだけを右側へ寄せるために使用しています。

object-fit

画像を指定した枠へどのように収めるかを決めます。

- **cover** : 枠全体を埋める。画像の一部が切れる場合がある
- **contain** : 画像全体を表示する。枠内に余白ができる場合がある

現在は体型画像に **cover**、参考画像プレビューに **contain** を使っています。

:focus

入力欄が選択され、文字を入力できる状態に適用される疑似クラスです。

```
.chatInput:focus {
  border-color: var(--accent);
}
```

::placeholder

入力前に表示する例文の見た目を指定する疑似要素です。

```
.chatInput::placeholder {  
  color: var(--muted);  
}
```

:disabled

操作できない状態の要素に適用される疑似クラスです。

```
.chatSendButton:disabled {  
  opacity: 0.4;  
}
```

現在まだ実装していないこと

- 保存した目標体型を再読み込み後の選択表示へ反映する処理
- 参考画像の永続保存
- 参考画像のサーバーへのアップロード
- 身体写真のAI分析
- トレーニング記録
- AIメニュー作成
- OpenAIが作った回答を **reply** としてフロントエンドへ返す処理
- AIが他機能の共通データを取得するTool

AIチャットのフロントエンドを確認するときは、まず **AiChatSection.jsx** で入口を確認し、次に **app/chat/page.jsx** を上から順番に読みます。